에러 처리 및 예외 상황 관리 분석

개요

Oasis CLI는 Go 언어의 관용적인 에러 처리 패턴을 기반으로 하여 계층적이고 체계적인 에러 관리 시스템을 구현하고 있습니다. 이 문서는 CLI 전반에 걸친 에러 처리 전략, 사용자 친화적 메시지 생성 과정, 그리고 복구 가능한 에러의 처리 방식을 상세히 분석합니다.

계층별 에러 처리 전략

1. 최상위 계층 - 메인 진입점

main.go의 에러 처리

```
func main() {
if err := cmd.Execute(); err != nil {
os.Exit(1)
}
}
```

특징:

- 최상위에서 모든 에러를 포착하여 프로그램 종료 코드 설정
- 단순하고 명확한 에러 처리로 예측 가능한 동작 보장
- Cobra 프레임워크의 에러 처리 메커니즘과 통합

2. 명령어 계층 - Cobra 명령어 처리

cobra.CheckErr() 패턴

CLI 전반에서 일관되게 사용되는 에러 처리 패턴:

```
// 설정 로딩 시 에러 처리
err := config.Load(v)
cobra.CheckErr(err)

// 네트워크 연결 시 에러 처리
conn, err := connection.Connect(ctx, npa.Network)
cobra.CheckErr(err)

// 파일 읽기 시 에러 처리
rawTx, err := os.ReadFile(filename)
cobra.CheckErr(err)
```

장점:

- 일관된 에러 처리 방식으로 코드 가독성 향상
- Cobra의 내장 에러 출력 기능 활용
- 에러 발생 시 즉시 프로그램 종료로 안전성 보장

3. 비즈니스 로직 계층 - 도메인별 에러 처리

설정 관리 에러 처리

```go
func (cfg *Config) Validate() error {
if err := cfg.Networks.Validate(); err != nil {
return fmt.Errorf("failed to validate network configuration: %w", err)
}
if err := cfg.Wallet.Validate(); err != nil {
return fmt.Errorf("failed to validate wallet configuration: %w", err)
}
return nil
}
```

특징:

- 에러 래핑(error wrapping)을 통한 컨텍스트 정보 보존
- 계층적 검증으로 구체적인 에러 위치 파악 가능
- 사용자에게 의미 있는 에러 메시지 제공

지갑 관리 에러 처리

```go
func (af *fileAccountFactory) Load(name string, passphrase string, rawCfg map
// 설정 검증
var cfg wallet.AccountConfig
if err := cfg.UnmarshalMap(rawCfg); err != nil {
return nil, err
}
```

```
// 파일 로딩
raw, err := os.ReadFile(getAccountFilename(name))
if err != nil {
return nil, fmt.Errorf("failed to load account state: %w", err)
}

// 암호화된 상태 해독
var state *secretState
if state, err = envelope.Open(passphrase); err != nil {
return nil, fmt.Errorf("failed to open account state (maybe incorrect passphras
}

return newAccount(state, &cfg)
}
```

특징:

- 단계별 에러 처리로 정확한 실패 지점 식별
- 사용자 친화적 힌트 제공 (“maybe incorrect passphrase?”)
- 에러 컨텍스트 보존을 통한 디버깅 지원

사용자 친화적 에러 메시지 생성 과정

1. 에러 메시지 향상 시스템

PrettyErrorHints 함수

```
func PrettyErrorHints(
_ context.Context,
npa *NPASelection,
_ connection.Connection,
_ interface{},
_ interface{},
failedRes *types.FailedCallResult,
) string {
errMsg := failedRes.Error()
// Testnet Sapphire 특정 에러에 대한 힌트
if npa != nil && npa.ParaTime != nil &&
npa.Network.ChainContext == config.DefaultNetworks.All["testnet"].ChainContext &
npa.ParaTime.ID == config.DefaultNetworks.All["testnet"].ParaTimes.All["sapphire
(failedRes.Module == accounts.ModuleName && failedRes.Code == 2 ||
failedRes.Module == core.ModuleName && failedRes.Code == 5) {
errMsg += "\nTip: You can get TEST tokens at https://faucet.testnet.oasis.io or
}
// 스테이킹 관련 에러 힌트
if failedRes.Module == staking.ModuleName {
if failedRes.Code == 5 {
errMsg += "\nTip: Did you forget to run `oasis account allow`?"
```

```
} else if failedRes.Code == 9 {
errMsg += "\nTip: You can see minimum staking transfer amount by running `oasis
}
}
return errMsg
}
```

특징:

- 컨텍스트 기반 힌트 제공 (네트워크, ParaTime 정보 활용)
- 실행 가능한 해결책 제시
- 외부 리소스 링크 제공 (faucet, Discord)

2. 검증 에러 메시지

주소 검증 에러

```
func CheckAddressNotReserved(cfg *config.Config, address string) error {
if address == rewards.RewardPoolAddress.String() {
return fmt.Errorf("address '%s' is ParaTime rewards pool address", address)
}
if address == accounts.CommonPoolAddress.String() {
return fmt.Errorf("address '%s' is ParaTime common pool address", address)
}
// ParaTime 네이티브 주소 검사
for netName, net := range cfg.Networks.All {
```

```
for ptName, pt := range net.ParaTimes.All {
if types.NewAddressFromConsensus(staking.NewRuntimeAddress(pt.Namespace())).Str
return fmt.Errorf("did you mean --paratime %s? Address '%s' is native address o
ptName, address, ptName, netName)
}
}
}
return nil
}
```

특징:

- 구체적인 에러 원인 설명
- 대안 제시 ("did you mean --paratime %s?")
- 컨텍스트 정보 포함 (네트워크, ParaTime 이름)

3. 계정 존재 확인 에러

```
func checkAccountExists(cfg *config.Config, name string) {
if _, exists := cfg.Wallet.All[name]; exists {
cobra.CheckErr(fmt.Errorf("account '%s' already exists in the wallet", name))
}
if _, exists := cfg.AddressBook.All[name]; exists {
cobra.CheckErr(fmt.Errorf("address named '%s' already exists in the address book
```

```
}

}
```

특징:

- 명확한 충돌 원인 식별
- 지갑과 주소록 모두 확인하여 포괄적 검증

복구 가능한 에러의 처리 방식

1. 강제 실행 옵션 (-force)

CheckForceErr 함수

```
func CheckForceErr(err interface{}) {
// 에러가 없는 경우
if err == nil {
return
}

// --force 옵션이 제공된 경우
if IsForce() {
fmt.Printf("Warning: %s\nProceeding by force as requested\n", err)
return
}
```

```
// --force 힌트와 함께 에러 출력 후 종료

errMsg := fmt.Sprintf("%s", err)

errMsg += "\nUse --force to ignore this check"

cobra.CheckErr(errMsg)

}
```

특징:

- 사용자가 위험을 인지하고 강제 실행 가능
- 경고 메시지로 위험성 알림
- 명확한 해결 방법 제시

2. 설정 마이그레이션

자동 마이그레이션 시스템

```
func (cfg *Config) migrateVersions() (bool, error) {

var changes bool

for v := cfg.LastMigration; v < latestMigrationVersion; v++ {

fn, ok := migrations[v]

if !ok {

return false, fmt.Errorf("missing migration from version %d", v)

}


ch, err := fn(cfg)

if err != nil {
```

```
return false, fmt.Errorf("failed to apply migration from version %d: %w", v, err
}
changes = changes || ch
cfg.LastMigration = v + 1
}
return  changes, nil
}
```

특징:

- 버전별 순차적 마이그레이션
- 실패 시 구체적인 버전 정보 제공
- 변경 사항 추적으로 불필요한 저장 방지

네트워크 설정 마이그레이션

```
func (cfg *Config) migrateNetworks() bool {
var  changes  bool
for  name, net := range  cfg.Networks.All {
defaultCfg, knownDefault := Default.Networks.All[name]
oldNetwork, knownOld := OldNetworks[name]
if !knownDefault || !knownOld {
continue
}

// 체인 컨텍스트 업데이트
```

```
for _, oldChainContext := range oldNetwork.ChainContexts {
if net.ChainContext == oldChainContext {
net.ChainContext = defaultCfg.ChainContext
changes = true
break
}
}
// RPC 엔드포인트 업데이트
for _, oldRPC := range oldNetwork.RPCs {
if net.RPC == oldRPC {
net.RPC = defaultCfg.RPC
changes = true
break
}
}
}
return changes
}
```

특징:

- 기존 설정과 새 기본값 비교
- 알려진 구 설정 자동 업데이트
- 사용자 개입 없이 투명한 마이그레이션

3. 트랜잭션 처리 에러 복구

오프라인 모드 지원

```
func PrepareConsensusTransaction(ctx context.Context, npa *NPASelection, signer

// 온라인 모드에서 가스 추정

if !txOffline && gas == invalidGasLimit {

gas, err = conn.Consensus().Core().EstimateGas(ctx, &consensus.EstimateGasReques

Signer: signer.Public(),

Transaction: tx,

})

if err != nil {

return 0, nil, fmt.Errorf("failed to estimate gas: %w", err)

}

}


// 오프라인 모드에서 필수 값 검증

if tx.Nonce == invalidNonce || tx.Fee.Gas == invalidGasLimit {

return 0, nil, fmt.Errorf("nonce and/or gas limit must be specified in offline

}

return gas, fee, nil

}
```

특징:

- 네트워크 연결 실패 시 오프라인 모드로 대체
- 필수 매개변수 명시적 요구
- 명확한 에러 메시지로 사용자 가이드

에러 처리 패턴 분석

1. 일관된 에러 래핑 패턴

```
// 컨텍스트 정보와 함께 에러 래핑
return fmt.Errorf("failed to validate network configuration: %w", err)
return fmt.Errorf("failed to load account state: %w", err)
return fmt.Errorf("failed to apply migration from version %d: %w", v, err)
```

장점:

- 에러 체인 보존으로 근본 원인 추적 가능
- 계층별 컨텍스트 정보 누적
- Go 1.13+ 에러 래핑 기능 활용

2. 조건부 에러 처리

```
// 온라인/오프라인 모드에 따른 다른 처리
if !txOffline && nonce == invalidNonce {
nonce, err = conn.Runtime(npa.ParaTime).Accounts.Nonce(ctx, client.RoundLatest,
if err != nil {
return 0, nil, "", fmt.Errorf("failed to query nonce: %w", err)
```

```
}

}


if nonce == invalidNonce {

return 0, nil, "", fmt.Errorf("nonce must be specified in offline mode")

}
```

특징:

- 실행 컨텍스트에 따른 적응적 에러 처리
- 대체 실행 경로 제공
- 명확한 요구사항 명시

3. 검증 기반 에러 방지

```
// 트랜잭션 타입별 검증

switch dtx := tx.(type) {

case *consensusTx.SignedTransaction, *types.UnverifiedTransaction:

// 서명된 트랜잭션, 바로 브로드캐스트

sigTx = tx

case *consensusTx.Transaction:

// 서명되지 않은 합의 트랜잭션, 먼저 서명

acc := common.LoadAccount(cfg, npa.AccountName)

sigTx, err = common.SignConsensusTransaction(ctx, npa, acc, conn, dtx)

cobra.CheckErr(err)
```

```
}
```

특징:

- 타입 안전성을 통한 런타임 에러 방지
- 각 케이스별 적절한 처리 로직
- 컴파일 타임 검증 활용

에러 로깅 및 디버깅 지원

1. 상세한 에러 컨텍스트

```
// 파일 작업 에러
if err := os.WriteFile(getAccountFilename(name), raw, 0o600); err != nil {
return nil, fmt.Errorf("failed to save state: %w", err)
}

// 네트워크 작업 에러
chainCtx, err := conn.Consensus().Core().GetChainContext(ctx)
if err != nil {
return 0, nil, fmt.Errorf("failed to query chain context: %w", err)
}
```

특징:

- 작업 유형과 실패 지점 명시

- 원본 에러 정보 보존
- 디버깅에 필요한 충분한 정보 제공

2. 사용자 액션 가이드

```
// 트랜잭션 서명 전 확인
fmt.Printf("You are about to sign the following transaction:\n")
PrintTransaction(npa, tx)
Confirm("Sign this transaction?", "signing aborted")
fmt.Println("(In case you are using a hardware-based signer you may need to conf
```

특징:

- 사용자 확인 절차 포함
- 하드웨어 지갑 사용 시 추가 안내
- 명확한 중단 메시지

보안 관련 에러 처리

1. 민감한 정보 보호

```
// 비밀번호 관련 에러에서 구체적 정보 숨김
if  state, err = envelope.Open(passphrase);  err != nil {
return  nil, fmt.Errorf("failed to open account state (maybe incorrect passphras
}
```

특징:

- 구체적인 실패 원인 숨김으로 보안 강화
- 사용자에게 유용한 힌트만 제공
- 브루트 포스 공격 방지

2. 권한 검증 에러

```
func (sf *accountEntitySignerFactory) EnsureRole(role signature.SignerRole) erro
if  role != signature.SignerEntity {
return  signature.ErrInvalidRole
}
return  nil
}
```

특징:

- 명확한 권한 검증
- 표준 에러 타입 사용
- 보안 정책 강제

성능 관련 에러 처리

1. 리소스 관리

```
// 파일 핸들 안전한 종료
outputFile, err := os.Create(txOutputFile)
```

```
if err != nil {
cobra.CheckErr(fmt.Errorf("failed to open output file: %w", err))
}
defer outputFile.Close()
```

특징:

- defer를 통한 안전한 리소스 해제
- 에러 발생 시에도 리소스 누수 방지
- 명확한 에러 메시지

2. 타임아웃 처리

```
// 컨텍스트 기반 타임아웃
ctx, cancel := context.WithCancel(ctx)
defer cancel()

ch, err := conn.Runtime(pt).WatchEvents(ctx, []client.EventDecoder{decoder}, fal
cobra.CheckErr(err)
```

특징:

- 컨텍스트를 통한 취소 가능한 작업
- 리소스 정리 보장
- 무한 대기 방지

결론

Oasis CLI의 에러 처리 시스템은 다음과 같은 특징을 가집니다:

강점

1. **일관성**: cobra.CheckErr() 패턴으로 통일된 에러 처리
2. **사용자 친화성**: 컨텍스트 기반 힌트와 해결책 제공
3. **복구 가능성**: --force 옵션과 자동 마이그레이션 지원
4. **보안성**: 민감한 정보 보호와 권한 검증
5. **디버깅 지원**: 상세한 에러 컨텍스트와 체인 보존

개선 가능한 영역

1. **구조화된 로깅**: 현재는 단순한 출력 위주
2. **에러 분류**: 에러 타입별 체계적 분류 부족
3. **복구 전략**: 더 다양한 자동 복구 메커니즘 필요
4. **사용자 경험**: 대화형 에러 해결 가이드 부족

전반적으로 Oasis CLI는 Go 언어의 관용적 패턴을 잘 활용하여 안정적이고 사용자 친화적인 에러 처리 시스템을 구현하고 있습니다.